API Lifecycle DevOps Project Report

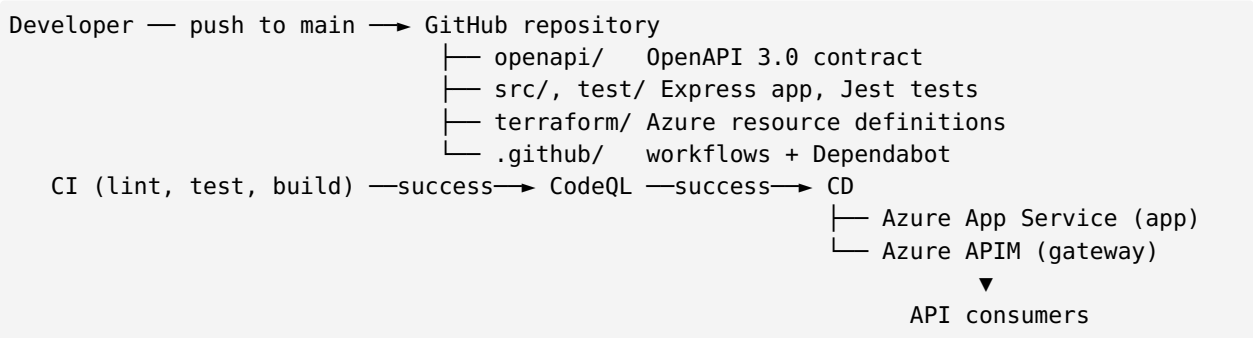
Padmalaya Moharana (moharana@kth.se) · Bogdan-Laurentiu Stefanescu (blst@kth.se)

Introduction

This project demonstrates an end-to-end DevOps workflow for the lifecycle of a REST API, using an API-first approach. A small Node.js/Express weather API (`GET /weather/{city}`) is defined by an OpenAPI 3.0 contract, which is automatically validated, tested, security-scanned and deployed to Azure App Service and Azure API Management through GitHub Actions, with the Azure infrastructure defined in Terraform. The application deliberately returns fixed sample data, so that the focus stays on the delivery process rather than business logic.

1. Architecture and processes

The contract, application, tests, infrastructure definitions and workflows are kept in one GitHub repository.



Development platform. GitHub hosts the repository and provides the platform features the pipeline relies on: Git version control and history, GitHub Actions for automation, encrypted secrets for the Azure credentials, the security tab where CodeQL results are reported, and Dependabot pull requests for dependency updates.

Continuous integration. `ci.yml` runs on every push to `main` (and on manual dispatch). It installs lockfile-pinned dependencies with `npm ci`, validates the OpenAPI file with Redocly, applies style rules with Spectral, runs the Jest/Supertest test against the Express app, and runs the build script. Any failing step fails the job and stops the chain.

Quality and security automation. `codeql.yml` is triggered by a successful CI run and performs static analysis of the JavaScript code, publishing results to GitHub's security tab. Dependabot (`dependabot.yml`) checks npm packages and GitHub Actions versions weekly and opens update pull requests. Spectral and Redocly act as quality gates on the API contract itself.

Continuous deployment. `deploy-apim.yml` is triggered by a successful CodeQL run. It re-runs the tests, authenticates to Azure using the `AZURE_CREDENTIALS` secret, deploys the application to App Service, imports the OpenAPI file into Azure API Management (APIM), and points the APIM API at the App Service backend.

Infrastructure as Code. Terraform declares the resource group, the APIM instance, a Linux App Service plan and a Node 20 Web App, with outputs for the gateway and Web App URLs. The resource group name and region are variables. Terraform is run manually before deployment; the workflows assume the resources exist.

Local demonstration. `scripts/local-ci-cd.ps1` runs the same lint, test and build steps locally, followed by the Azure CLI commands for App Service deployment, APIM import

and backend configuration, stopping at the first failing command. This makes the pipeline reproducible and easier to debug outside GitHub Actions.

2. Key design decisions

API-first with OpenAPI. A machine-readable contract can be linted automatically and imported directly into APIM, so the published interface comes from the same file developers review. The cost is possible drift between contract and implementation, since no test compares the two mechanically.

Node.js and Express. A small Express app is quick to build and fits naturally with the Node-based tooling (Spectral, Redocly, Jest). Separating `app.js` from `server.js` lets tests import the app without starting a server.

Repository structure. Contract, code, tests, infrastructure and workflows are versioned together, so a single commit can change the API and its delivery process consistently, and every change is reviewable in Git history.

GitHub Actions with three chained workflows. Keeping automation next to the code makes it versioned and reviewable. Splitting CI, CodeQL and CD into separate workflows makes each stage visible in the Actions tab, at the price of a less direct `workflow_run` chain that depends on exact workflow names.

Terraform and managed Azure services. Terraform makes the infrastructure reproducible and reviewable instead of being created by hand in the portal. App Service and APIM remove server management and let us demonstrate gateway publication.

Quality and security tools. Redocly and Spectral are standard, free OpenAPI linters that run directly in CI. CodeQL and Dependabot are built into GitHub, so they add security analysis and dependency updates without extra accounts, services or configuration outside the repository.

Small, deterministic tests. Supertest exercises the Express app in-process without deploying, which keeps CI fast and reliable.

3. Component interaction

The OpenAPI file links the stages: it is validated in CI, it defines the API imported into APIM, and the Express route implements it. A change flows through the system as follows:

1. A developer pushes a change to the code, contract, tests or infrastructure definitions to `main`.
2. CI installs dependencies, validates and lints the contract, tests the Express route and runs the build.
3. A successful CI run triggers CodeQL analysis of the source code.
4. A successful CodeQL run triggers deployment: tests run again, the workflow logs in to Azure and deploys the app to App Service.
5. The contract is imported into APIM and the API’s backend is set to the App Service URL.
6. Consumers call the APIM gateway, which forwards requests to the application.

Because each workflow only proceeds when the previous one succeeded, nothing is deployed unless both validation and security analysis passed. Terraform sits outside this chain: it provisions the App Service and APIM instances that the deployment workflow targets by name.

4. Limitations and trade-offs

- **Minimal application and tests:** fixed sample data, one route, one happy-path test; the `400`/`404` responses in the contract are not implemented or tested, and `npm run build` is only a placeholder.
- **No contract testing:** the implementation is not verified against the OpenAPI schema, so contract and code can drift.
- **CI trigger:** CI runs on pushes to `main` only, not on pull requests, so changes are validated after merge rather than before.
- **IaC outside the pipeline:** Terraform is not planned or applied by GitHub Actions, there is no remote state, and resource names are duplicated between Terraform and the deployment workflow instead of being passed through outputs.
- **Workflow chaining:** `workflow_run` keeps stages separate and visible, but is harder to follow than one workflow with dependent jobs.
- **Security scope:** CodeQL static analysis does not prove the application is secure and does not replace runtime or manual security testing.
- **Single environment:** no staging, approvals or rollback.
- **Credentials:** deployment uses a long-lived service principal secret; OIDC federation would be safer.
- **Configuration consistency:** the resource group, APIM name and Web App name must match across Terraform, the workflow and GitHub secrets; a mismatch breaks deployment.
- **Gateway and operations:** no APIM policies (keys, rate limits, auth) and no monitoring or alerting.
- **Cloud cost:** managed services reduce operational work but APIM and App Service are paid, always-on resources, and the APIM Developer tier is slow to provision.

Conclusion

The project demonstrates a connected API lifecycle: an OpenAPI contract is linted, an Express implementation is tested, CI gates subsequent CodeQL analysis, and successful security analysis gates deployment to Azure App Service and API publication through APIM. Terraform describes the cloud resources, and a local PowerShell script illustrates the same checks and deployment operations. The key design benefit is the separation of concerns while keeping the contract, application, infrastructure, and automation versioned together. The principal trade-off is scope: this is a compact educational proof of concept, not a complete production platform. In particular, pull-request validation, automated infrastructure provisioning, stronger contract and integration tests, managed identity, gateway policies, and multi-environment release controls would be valuable next steps.